Quoridor Game

Artificial intelligence (CSE472s)

Team Members

Name	ID
Yousef Mahmoud Mohamed	2100994
Moaz Ragab Abuelmagd	2100938
Youssef Ashraf Mohammed	2201056
Ahmed Ashraf Ali	2100255
Omar Ayman Ahmed	2100728

Project zip file: Download from here
GitHub Repo: AshrafByte/quoridor-ai-bot

Video Link: View from here

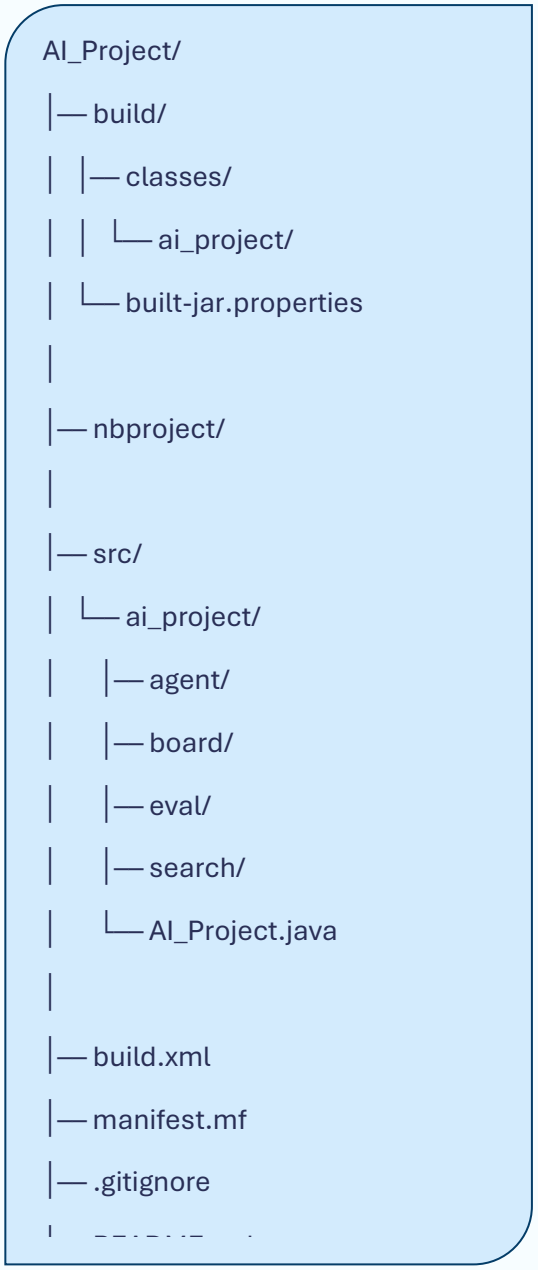
1. Design Decision

1. Overall Architecture

The Quoridor AI Project follows a layered software architecture that separates core game logic, AI decision-making, and user interaction. This separation ensures modularity, ease of maintenance, and flexibility in experimenting with different AI strategies.

The system is divided into the following layers:

- User Interface Layer
- Game Engine Layer
- Core Game Model
- AI Module



2. Core Game Model

The Core Game Model represents the complete state of the Quoridor game and enforces all official game rules.

Responsibilities:

- Represent the 9×9 board grid
- Track pawn positions and remaining walls
- Validate legal moves and wall placements
- Prevent illegal wall blocking
- Detect terminal states and winning conditions

Main Components:

- Board – stores grid structure and wall placements
- GameState – holds current player state and move history
- Move – represents pawn moves or wall placements
- RuleChecker – validates legality of moves

This module acts as the single source of truth for the game.

3. AI Module

The AI Module determines optimal moves for computer-controlled players. It interacts only with the game state and does not depend on user interface logic.

Responsibilities:

- Analyse possible game states
- Evaluate positions using heuristics
- Select the best move using search algorithms

Supported AI Techniques:

- Minimax with Alpha-Beta Pruning
- Monte Carlo Tree Search (optional)
- Heuristic Evaluation Functions

A strategy interface allows AI algorithms to be swapped without modifying other components.

4. Game Engine

The Game Engine manages the overall game flow and turns progression.

Responsibilities:

- Alternate turns between players
- Request moves from human players or AI agents
- Apply validated moves to the game state
- Check for end-game conditions

The engine serves as a controller that connects the interface, AI, and core model.

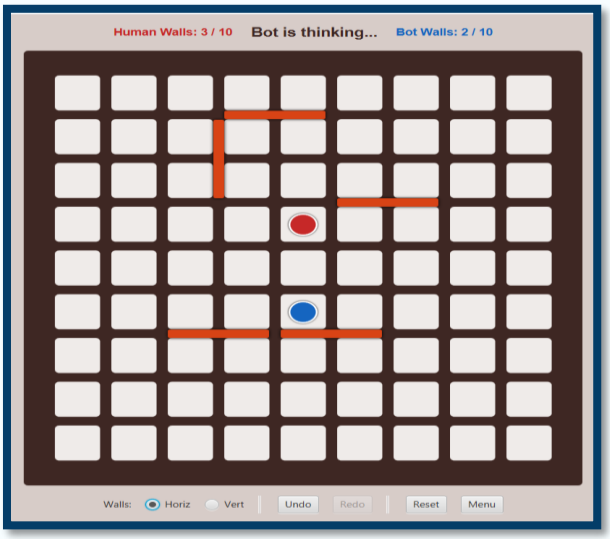
5. Interface Layer

The Interface Layer handles all interactions with the user.

Responsibilities:

- Read user input
- Display the board and game state
- Support console-based or graphical interfaces

This layer contains no game logic and relies entirely on the game engine.



6. System Workflow

1. Game Engine requests a move
2. Human or AI player provides a move
3. Move is validated by the Core Game Model
4. Game state is updated
5. Win condition is checked
6. Next turn begins or game ends

7. Design Advantages

- Clear separation of concerns
- Easy replacement of AI strategies
- Improved testability
- Scalable and maintainable structure
- Reusable game logic

8. Bonus

- Undo-Redo
- Ai difficulty levels
- Algorithm documentation for each difficulty level

2. Implementation challenges and solutions

1. Wall representation and rule enforcement.

- **Challenge:** Modelling walls so they correctly block movement, prevent overlaps/crossings, and still guarantee at least one path for each player was tricky.
- **Solution:** Represented walls as structured data attached to board edges and used BFS shortest path checks after each tentative wall placement to reject illegal placements.

2. Correct pawn movement (jumps and diagonals)

- **Challenge:** Implementing the Quoridor movement rules (orthogonal moves, jumping over an adjacent pawn, and diagonal moves when a jump is blocked) without missing edge cases.
- **Solution:** Decomposed movement into simple directional checks, then added a clear second stage for "if adjacent to opponent ⇒ try jump ⇒ else try diagonals," with unit tests on small board scenarios to validate behaviour.

3. Keeping AI separate from game logic

- **Challenge:** Avoiding a "god class" where the AI directly manipulates board internals or reimplements rules.
- **Solution:** Defined a narrow board interface (getLegalMoves, applyMove, isTerminal, getWinner, shortestPathLength) and made the AI depend only on that interface, so all rules remain inside the board module.

4. Search performance and complexity

- **Challenge:** Naive minimax over full Quoridor move space (especially walls) quickly becomes too slow.
- **Solution:** Used depth-limited minimax with alpha–beta pruning, tuned search depth per difficulty level, and based the evaluation on shortest path lengths instead of deeper lookahead when needed.

5. Managing game state during search

- **Challenge:** Mutating a single board instance during minimax caused hard-to-debug state corruption and undo bugs.
- **Solution:** Switched to an immutable style applyMove that returns a new board state for each move, which simplified the search code and removed the need for explicit undo operations, at the cost of a small, acceptable performance overhead

3. AI Algorithm explanation

1. Key Achievements

- Successfully implemented Minimax search with alpha-beta pruning optimization
- Developed a multi-component evaluation function incorporating path analysis, tactical awareness, and strategic planning
- Created three distinct difficulty levels with measurable performance differences
- Achieved intelligent wall placement and pawn movement coordination
- Implemented "trap detection" through path redundancy analysis

2. Algorithm Overview

2.1 Core Algorithm: Minimax with Alpha-Beta Pruning

The AI uses the **Minimax algorithm**, a decision-making algorithm for two-player zero-sum games. The algorithm explores the game tree by alternating between maximizing the AI's score and minimizing the opponent's score.

Alpha-Beta Pruning Enhancement: Alpha-beta pruning is applied to eliminate branches that cannot influence the final decision, significantly reducing the number of nodes evaluated without affecting the result.

2.2 Algorithm Selection Rational

Why Minimax?

- **Perfect for two-player games:** Quoridor is a deterministic, perfect-information, zero-sum game
- **Guarantees optimal play:** Within the search depth limit, finds the best possible move
- **Well-understood and debuggable:** Established algorithm with proven correctness
- **Efficient with pruning:** Alpha-beta pruning reduces complexity dramatically

Alternative Algorithms Considered:

- **Monte Carlo Tree Search (MCTS):** Better for games with higher branching factors, but unnecessary for Quoridor

3. Core Components

3.1 Search Strategy Architecture

1. AIBot (Agent)
2. MinimaxSearch
3. PathLengthEvaluation (Evaluation Function)
4. Board State (Game Logic)

3.2 Component Responsibilities

- **AIBot:** Manages difficulty settings and search depth. Implements "mistake injection" for Easy mode. Coordinates between search strategy and board state.
- **MinimaxSearch:** Implements recursive minimax algorithm with alpha-beta pruning. Handles tie-breaking through random selection. Manages search depth and terminal state detection.
- **PathLengthEvaluation:** Multi-factor evaluation function. Combines tactical and strategic considerations. Implements "trap detection" heuristic.
- **Board:** State representation and movement generation. Wall-aware pathfinding using BFS. Move validation and legality checking.

4. Implementation Details

4.1 Minimax Search with Alpha-Beta Pruning

Key Implementation Features

1. **Tie-Breaking Strategy:** When multiple moves have equal scores, randomly select one. Prevents predictable behavior. Uses epsilon comparison (1e-9) for floating-point equality.
2. **Alpha-Beta Pruning:** Alpha represents the best score maximizer can guarantee. Beta represents the best score minimizer can guarantee. Prune when $\beta \leq \alpha$ (current branch cannot affect final decision).
3. **Depth-Limited Search:** Search stops at specified depth or terminal state. Prevents excessive computation time. Depth varies by difficulty level.

4.2 Evaluation Function

The evaluation function is the "brain" of the AI, assessing how favorable a board position is. Our implementation uses **five key factors:**

Component 1: Terminal State Detection

This component ensures the AI immediately recognizes a game-ending state (win, loss, or draw).

- **Purpose**: To assign the highest possible priority to immediate game outcomes.

- **Scoring**:
  - An immediate win for the AI receives an extremely high positive score.
  - An immediate loss for the AI receives an extremely high negative score.
  - A draw receives a neutral score.

Component 2: Base Path Difference Score

This is the fundamental strategic heuristic, prioritizing the shortest path to the goal.

- **Calculation**: The score is determined by the difference between the player's shortest path length and the AI's shortest path length.
- **Purpose**: To prefer positions where the AI is closer to the goal line than the opponent.
- **Trap Detection (Critical):** If a shortest-path algorithm (like Breadth-First Search) reveals that the AI is completely blocked (distance is infinity), a heavy negative penalty is applied. Conversely, if the opponent is completely blocked, a heavy positive bonus is applied.

Component 3: Tactical

This component introduces tactical awareness by detecting and prioritizing critical, immediate win/loss threats.

- **Kill Move**: Adds a significant positive bonus if the AI is within immediate winning range.
  - If my pawn is about one move from victory, a very large bonus is added.
  - If my pawn is very close, a smaller, but still high, bonus is added.
- **Panic**: Applies a significant negative penalty if the opponent is about to win.
- **Purpose**: To override the base path difference score in critical situations, prioritizing immediate winning moves and blocking the opponent's immediate victory.

Component 4: Path Redundancy

This is an advanced strategic component that assesses the quality of the shortest path, not just its length, by counting the number of moves that reduce the player's distance to the goal.

- **Calculation**: The score is increased for each "good move" (a pawn move that shortens the path to the goal) available to the AI and decreased for each good move available to the opponent.
- **Purpose**: To prefer positions with multiple available forward-progress options. This indicates a "wide" or redundant path that is harder for the opponent to block completely.
- **Strategic Importance**: This is a key innovation for the higher difficulty levels. A path with only one good move is highly vulnerable to being trapped with a single wall. Multiple good moves indicate a safer, more resilient path.

Component 5: Wall Conservation

This component introduces a slight bias toward preserving wall resources.

- **Calculation**: A very small negative penalty is applied for each wall the AI has already used.
- **Purpose**: To serve as a minor tie-breaker. Walls are valuable for both offense and defense; all else being equal, the AI slightly prefers the move that leaves it with more wall resources for future flexibility.
- **Weight Rationale**: The small factor ensures this component never overrides any strategic or tactical consideration, only affecting the choice between otherwise equal moves.

4.3 Pathfinding with Wall Awareness

The shortest path calculation is critical for evaluation accuracy. We use **Breadth-First Search (BFS)** with wall detection.

Key Features

1. **Wall-Aware Navigation:** IS-EDGE-BLOCKED checks if a wall prevents movement. Accounts for both horizontal and vertical walls. Ensures paths are traversable.
2. **Goal Detection:** Player 1 goal: Any cell in row 8 (bottom). Player 2 goal: Any cell in row 0 (top). BFS naturally finds shortest path to any goal cell.
3. **Infinity Handling:** Returns infinity if no path exists. Evaluation function handles this as severe penalty.

4.4 Move Generation

Move generation produces all legal moves in a specific order: **pawn moves first, then wall placements.**

Branching Factor Analysis

Game Phase	Pawn Moves	Wall Placements	Total Branching Factor
Early Game	2-4	100-120	~100-120
Mid Game	2-4	40-80	~50-80
Late Game	2-4	10-40	~20-40

Special Move Handling: Jumps

When two pawns face each other with no intervening squares, special jump rules apply:

Jump Rules:

1. Can jump straight over opponent if space is available
2. If wall blocks straight jump, can jump diagonally
3. Cannot overlap pawns on same square

4.5 Difficulty Implementation

Three difficulty levels are implemented through depth variation and mistake injection:

Easy Mode (Depth = 1)

Characteristics:

- **Depth 1:** Only looks at immediate consequences (greedy)
- **30% mistake rate:** Occasionally makes random moves
- **Behavior:** Predictable, makes obvious tactical errors

Medium Mode (Depth = 2)

Characteristics:

- **Depth 2:** Looks one move ahead for both players
- **No mistakes:** Always plays optimally within depth limit
- **Behavior:** Competent tactical play, some strategic limitations

Hard Mode (Depth = 3)

Characteristics:

- **Depth 3:** Looks two moves ahead (AI → Opponent → AI)
- **Full evaluation:** Leverages path redundancy for trap detection
- **Behavior:** Strong tactical and strategic play, difficult to trap

5. Complexity Analysis

5.1 Time Complexity

Without Alpha-Beta Pruning:

- $O(b^d)$ Where b = branching factor, d = search depth

With Alpha-Beta Pruning:

- In the **best case** (ordering), alpha-beta reduces complexity to $O(b^{d/2})$.
- In **average case** we get $O(b^{3d/4})$

5.2 Evaluation Function time Complexity

Practical Interpretation:

- **Easy:** < 0.1 seconds per move
- **Medium:** < 1 second per move
- **Hard:** 1-5 seconds per move (depending on position)

6. Difficulty Levels Comparison

6.1 Comparison Table

Feature	Easy	Medium	Hard
Search Depth	1	2	3
Mistake Rate	30% (random)	Very low	~ 0%
Move Time	< 0.1 s	< 1 s	1–5 s
Strategic Planning	None	Limited	Strong
Trap Detection	No	Partial	full
Wall Usage	Random	Reactive	Strategic
Predicted Win Rate vs Human	Beginner-friendly	Challenging	Expert-level

6.2 Strategic Behaviour Differences

Easy Mode Behaviour

Characteristics:

- Makes greedy decisions (immediate gain only)
- 30% chance to make completely random move
- Doesn't anticipate opponent responses
- Often places walls ineffectively

Mistake

- At most it leads to opponent wins.

Medium Mode Behaviour

Characteristics:

- Considers immediate opponent response
- Makes tactically sound decisions
- Sometimes falls into strategic traps
- Decent wall placement

Limitation:

- At most it misses multi-move setups.

Hard Mode Behaviour

Characteristics:

- Plans 2+ moves ahead
- Uses path redundancy to avoid traps
- Coordinates wall placements strategically
- Rarely makes tactical errors
- It Avoids trap, maintains flexibility

7. summary

7.1 Achievements

- Robust Minimax Implementation
- Sophisticated Evaluation Function
- Three Distinct Difficulty Levels
- Efficient Pathfinding

7.2 Learning Outcomes

This project demonstrated several key AI concepts:

- Adversarial Search: Minimax algorithm for two-player games
- Optimization: Alpha-beta pruning for computational efficiency
- Heuristic Design: Multi-factor evaluation function development
- Strategic Reasoning: Balancing short-term tactics with long-term strategy
- Performance Tuning: Depth/quality tradeoffs for difficulty levels

7.3 Real-World Applications

The techniques used in this project generalize to:

- Chess engines: Similar minimax with position evaluation
- Game AI: Any turn-based strategy game
- Decision-making systems: Resource allocation, planning problems
- Competitive programming: Optimization and search algorithms

4. Assumptions

1. Game Rules Assumptions

- **Board Configuration**: The game is played on a standard 9×9 grid board
- **Wall Allocation**: Each player starts with exactly 10 walls
- **Starting Positions**: Player 1 starts at the top (row 0), Player 2 starts at the bottom (row 8)
- **Win Condition**: A player wins by reaching any cell on the opposite side of the board (Player 1 reaches row 8, Player 2 reaches row 0)
- **Turn Structure**: Players strictly alternate turns with no simultaneous moves

2. Path Validity Assumptions

- **Reachable Goal:** The game enforces that there must always remain at least one valid path to the goal for each player.
- **Wall Placement Validation**: The game logic (via isWallPlacementValid()) ensures no wall can be placed that completely blocks a player from reaching their goal
- **AI Benefit**: The AI never needs to handle the case where a player is completely trapped (distance = infinity) during normal gameplay
- **BFS Guarantee**: The pathfinding algorithm (BFS) is guaranteed to find a path, though it may be arbitrarily long
- **Evaluation Robustness**: The evaluation function includes handling for ∞ distance as a safety mechanism, but this should only occur in theoretical/test scenarios
- **Implementation Detail**: The evaluation function returns ±100,000 when a player has no path (distance = infinity), but the game rules prevent this scenario from occurring in actual gameplay. This defensive programming ensures the AI handles edge cases gracefully.

3. Move Generation and Validation Assumptions

- **Legal Move Correctness**: All moves returned by getLegalMoves(playerId) are assumed to be legal and valid according to game rules
- **Move Application**: The applyMove() function correctly generates new board states without side effects
- **Opponent Legality**: The opponent (whether human or AI) always makes legal moves validated by the game interface

- **No Invalid States**: The game engine prevents illegal wall overlaps, invalid pawn positions, or rule violations
- **UI Validation Layer**: The user interface performs validation before accepting moves, which ensures the AI never encounters illegal board states during search.

4. Algorithm Assumptions

1. BFS Pathfinding Assumptions

- **Shortest Path Guarantee**: BFS is assumed to always find the shortest path in terms of number of moves (which is correct for unweighted graphs)
- **Wall Detection**: The isEdgeBlocked() function accurately determines if a wall prevents movement between adjacent cells
- **Completeness**: Given that a path always exists (assumption 6.2), BFS will always terminate and return a valid distance

2. Minimax Search Assumptions

- **Deterministic Game**: Quoridor has no random elements, making Minimax applicable
- **Perfect Information**: Both players have complete knowledge of the board state
- **Rational Opponent**: The opponent is assumed to play optimally within the search depth (minimax assumption)
- **Static Evaluation**: Board positions can be accurately evaluated without search to terminal states

5. Performance and Resource Assumptions

The implementation assumes games finish within a reasonable timeframe:

- **Terminal States**: The game will eventually reach a terminal state (one player reaches their goal)
- **No Infinite Loops**: Players cannot create board states that loop indefinitely
- **Practical Game Length**: Most games conclude within 30-60 moves, preventing excessive computation
- **UI Responsiveness**: The game checks terminal conditions after each move using checkGameStatus()

6. Implementation Specific Assumptions

- **Immutable Board States**: applyMove() creates new board instances rather than modifying existing ones
- **No Side Effects**: Evaluation and search functions do not modify the board state they analyze
- **State Independence**: Each board state can be evaluated independently without global state dependencies
- **Tie-Breaking Randomness**: Java's Random class provides sufficient randomness for move selection among equally valued options
- **Easy Mode Randomness**: The 30% mistake probability uses rng.nextDouble() which is uniformly distributed
- **Seed Independence**: No specific random seed is set, allowing variation across different game sessions

5. References

1. Quoridor Game Rules

https://en.wikipedia.org/wiki/Quoridor

2. YouTube

https://www.bing.com/videos/riverview/relatedvideo?q=Quoridor+Game+%d8%b4%d8%b1%d8%ad&mid=62A4ED7C064F2A16D80662A4ED7C064F2A16D806&FORM=VIRE

3. Java

https://www.w3schools.com/java/

4. Game

https://play.google.com/store/apps/details?id=com.quoridor.gg.Quoridor